

Algorithmique avancée – Examen Réparti 1
UPMC — Master d’Informatique —
Novembre 2014 – durée 2h

Les seuls documents autorisés sont les polys de cours, ainsi que la copie double personnelle.

1 QCM [7 points]

Dans ce QCM, pour chaque question vous devez donner 1 seule réponse et expliquer votre choix par une ligne de texte ou une figure. Le barème sera le suivant : **Réponse correcte et correctement argumentée : 1 point.** **Réponse incorrecte ou correcte mais mal argumentée : 0 point.** *Il n’y a pas de points négatifs.*

Q1. Dans un tournoi binomial

- A) le parcours infixe donne les clés dans l’ordre croissant
- B) le parcours préfixe donne les clés dans l’ordre croissant
- C) chacune des branches est étiquetée de façon croissante

Q2. La récurrence $T(n) = 2T(n/2) + \Theta(1)$ a pour solution (et donner un exemple d’algorithme qui a cette complexité)

- A) $T(n) = \Theta(n \log n)$
- B) $T(n) = \Theta(n)$
- C) $T(n) = \Theta(\log n)$

Q3. L’arbre binomial B_k

- A) a une profondeur $\Theta(\log(k))$
- B) a ses fils gauche et droit qui sont des arbres binomiaux B_{k-1}
- C) a sa racine de degré (i.e. d’arité) k

Q4. Toute file binomiale

- A) a une taille qui est une puissance de 2
- B) est une file binomiale relâchée
- C) contient au moins deux tournois binomiaux

Q5. Tout arbre 2-3-4 contenant n clés

- A) a une hauteur en $\Theta(\log n)$
- B) est quasiment équilibré : deux sous-arbres d’un même nœud peuvent avoir des hauteurs qui diffèrent de 1
- C) a une racine d’arité $\Theta(\log(n))$

Q6. On considère une table de hachage de taille $m = 2n$ dans laquelle on insère n clés, avec une fonction de hachage uniforme. La proportion de cases vides dans la table vaut asymptotiquement lorsque $n \rightarrow \infty$

- A) $\exp(-1/2)$
- B) $2/\sqrt{n}$
- C) $1/2$

Q7. En utilisant un splay-tree de n clés, on peut réaliser un tri des n clés

- A) en temps moyen $O(n \log n)$
- B) en temps amorti $O(n \log n)$
- C) en temps pire $O(n \log n)$

2 Exercices [13 points]

Exercice 1

Donner un algorithme en $O(n \log n)$ comparaisons, qui étant donné une suite S de n nombres entiers et un nombre donné x , détermine s'il existe deux nombres p_1 et p_2 dans S tels que $p_1 + p_2 = x$.

Exercice 2

Montrer que le sous arbre gauche d'un arbre 2-3-4 contenant n clés peut contenir $\Theta(\sqrt{n})$ clés.

Exercice 3

On veut incrémenter un compteur binaire représenté par un tableau $A[0..k-1]$ de k bits (le tableau contenant le bit b_i en case i représente l'entier $\sum b_i 2^i$).

1) Écrire l'algorithme pour passer de x à $x+1$ dans le tableau ($0 \leq x < 2^k - 2$).

Quel est le nombre maximum de flips (passage d'un bit de 1 à 0 ou de 0 à 1) pour une opération d'incrément ?

2) On montre ici que le coût amorti, en nombre de flips, pour incrémenter le compteur de 0 à $n = 2^k - 1$ est de $O(1)$ pour une opération d'incrément.

- Méthode par agrégat : montrer que le nombre total de flips pour la suite des n incréments est $2n(1 - 1/n)$.

- Méthode de potentiel : on considère la fonction qui à un tableau associe son nombre de bits à 1. Montrer que c'est une fonction de potentiel. Quel est le coût amorti de l'opération d'incrément de i vers $i+1$?

Exercice 4

Décrire un algorithme de suppression

- soit pour le Hachage dynamique

- soit pour le Hachage extensible

en explicitant le nombre d'accès à la mémoire secondaire.

Exercice 5

Pour hacher les éléments d'un ensemble $\mathcal{U}$ dans une table d'adresses $[0..m-1]$, on considère la classe $\mathcal{H}$ des fonctions h_a , avec $a \in \{0, \dots, p-1\}$, où p est un nombre premier supérieur à m :

$$\forall x \in \mathcal{U}, h_a(x) = (ax \mod p) \mod m$$

Montrer que $\mathcal{H}$ n'est pas une classe de fonctions universelle.

On pourra utiliser l'exemple suivant où $m = 2$, $p = 7$ et les valeurs de hachage sont données dans la table :

x/a	0	1	2	3	4	5	6
1	0	1	0	1	0	1	0
2	0	0	0	1	1	1	1
3	0	1	0	0	1	1	0
4	0	0	1	1	0	0	1
5	0	1	1	1	0	0	0
6	0	0	1	0	1	0	1